

Documentation of the Automated Emergency Braking System

Papp Edina, Horváth Gábor Timót, Czéhmaster Anna, Dely Dániel, Fülöp Zoltán, Nagy Szabolcs, Nardai András

Széchenyi István University, Egyetem sq 1., Győr, 9026, Győr-Moson-Sopron, Hungary

1. Introduction

Az automatikus vészfék asszisztens (AEB – *Autonomous Emergency Braking*) egy fejlett vezetéstámogató funkció, amelynek célja az ütközések elkerülése vagy hatásának csökkentése automatikus fékbeavatkozás révén. A rendszer a Modern Autóipari Szoftverfejlesztés Alapjai nevű tárgy keretében valósult meg. A feladat egy valós ipari környezethez közel álló, ISO 26262 kompatibilis szoftverrendszer megtervezése és implementálása volt ROS2 segítségével.

A funkció lényege, hogy folyamatosan figyeli az ego jármű előtti területet, és ha veszélyes helyzetet észlel, automatikusan beavatkozik. Bemenetként szenzorból érkező objektum adatokat kap – pozíció, sebesség, gyorsulás –, valamint az ego jármű saját állapotát. Ezek alapján kiszámítja az ütközésig hátralévő időt és eldönti, hogy kell-e fékezni, és ha igen, milyen erősen. Kimenetként egy célsebességet ad ki a jármű fékrendszere felé, és jelzi, hogy az ütközés elkerülhető-e vagy sem.

A rendszer 0–50 km/h tartományban működik, 20 ms-es ciklusidővel. Három egymástól jól elkülöníthető szoftverkomponensre bontottuk: a **behavior planner** figyeli a jelenetet és kiválasztja a kritikus objektumot, a **plan long emergency** kiszámítja a lassulási profilt a beállított korlátok figyelembevételével, a **ctrl long emergency** pedig a trajectory alapján meghatározza és kiadja a célsebességet.

A projekt elkészültével a szoftver az egyetem tulajdonában lévő tesztautón kerül kipróbálásra.

2. Requirements

2.1. Item definition

Table 1: SYS-ITEM

ID	Leírás
ITEM-01	Az AEB funkció célja az ütközések elkerülése vagy enyhítése az objektumállapot alapján számított automatikus fékbeavatkozással.
ITEM-02	A funkció bemenete: objektumlista (pozíció, sebesség, gyorsulás), ego jármű állapot (sebesség, gyorsulás), vezetői fékjel.
ITEM-03	A funkció kimenete: jármű célsebesség, figyelmeztető jelzés (elkerülhető / nem elkerülhető).
ITEM-04	Működési tartomány: 0 - 50 km/h.
ITEM-05	Működési ciklusidő: 20 ms

2.2. Hazard Analysis and Risk Assessment

Table 2: HARA

Hazard ID	Veszély leírás	S	E	C	ASIL
H-01	AEB nem avatkozik be elkerülhető ütközés esetén	S3	E4	C3	ASIL D
H-02	AEB indokolatlan vészfékezést hajt végre	S2	E3	C3	ASIL C
H-03	Szenzoradat kiesés miatt hibás döntés	S3	E4	C3	ASIL D

2.3. Safety Goals

Table 3: SG

SG ID	Safety Goal	ASIL
SG-01	A rendszer elkerülhető ütközés esetén megfelelő időben generáljon fékbeavatkozást.	D
SG-02	A rendszer minimalizálja az indokolatlan beavatkozásokat.	D
SG-03	Szenzorhibák esetén biztosítson degradált, de biztonságos működést.	D

2.4. Functional Safety Requirements

Table 4: FSR

TSR ID	Követelmény	ASIL
FSR-01	A rendszer számítsa ki a szükséges lassulási profilt max. gyorsulás és jerk korlát mellett úgy, hogy az ego jármű a kritikus objektum mögött adott távolságban álljon meg.	D
FSR-02	A rendszer kezelje az alábbi objektumállapotokat: álló, lassuló, konstans sebességű, tolató objektum.	D
FSR-03	A rendszer különböztesse meg az elkerülhető és nem elkerülhető ütközést, és ezt külön strategy-ként jelezze.	D
FSR-04	A rendszer legyen robusztus max. 5 ciklusnyi adatkimaradás és $\pm 10\%$ zaj esetén az objektum pozíciója, sebessége és gyorsulása tekintetében.	D
FSR-05	A rendszer támogasson 3 ún. Soft Limit módot konfigurálható gyorsulás és jerk limitekkel.	C

2.5. Technical Safety Requirements

Table 5: TSR

TSR ID	Követelmény	ASIL
TSR-01	Szenzoradat validitás-ellenőrzés minden ciklusban.	D
TSR-02	Adatkimaradás esetén utolsó valid állapot extrapoláció max. 5 ciklusig.	D
TSR-03	Zajszűrés (pl. aluláteresztő szűrő vagy állapotbecslő).	D
TSR-04	Paraméterek külön kalibrációs fájlban tárolva (SW package része).	D
TSR-05	A rendszer támogasson 3 ún. Soft Limit módot konfigurálható gyorsulás és jerk limitekkel.	QM/ASIL C
TSR-06	Interfész definíció - bemenet: szenzor/objektum állapota: pozíció, sebesség és gyorsulás.	D
TSR-07	Interfész definíció - kimenet: célsebesség	D
TSR-08	Interfész definíció - belső kimenet: kritikus objektum állapotának információja: pozíció, sebesség és gyorsulás.	D
TSR-09	Interfész definíció - belső kimenet: stratégia: "noAction", "warningLevel", "longEmergencyAvoid", "longEmergencyImpact"	D

2.6. Software Safety Requirements

Table 6: SWE

SWE ID	Követelmény	ASIL
SWE-01	Determinisztikus futás fix ciklusidővel.	D
SWE-02	Lebegőpontos túlcsoordulás és numerikus stabilitás kezelése.	D
SWE-03	Hibadetektálás 0 értékű objektumtömb esetén.	D
SWE-04	Állapotgép implementálása (Monitoring / Warning / Partial Brake / Full Brake).	D
SWE-05	Konfigurálható paraméterek betöltése init során, integritásellenőrzéssel (CRC).	QM/ASIL C/D
SWE-06	Trace log generálása diagnosztikai célra.	C

2.7. Traceability

Table 7: Traceability

SG	FSR	TSR	SWE
SG-01	FSR-01, FSR-02, FSR-03	TSR-01, TSR-02, TSR-05	SWE-01, SWE-04
SG-02	FSR-02, FSR-03	TSR-01, TSR-03	SWE-04
SG-03	FSR-04	TSR-01, TSR-02, TSR-03	SWE-03

2.8. Paraméterek

Table 8: Paraméterek

Paraméter megnevezése	Jelentés	Mértékegység	Node
Érzékenység	Megadja, milyen hamar jelezzen illetve avatkozzon be a rendszer	Enum	behavior_planning
Érzékenységhez tartozó gyorsulás és jerk limitek	Megadja, hogy az egyes érzékenységi szintekhez mekkora gyorsulás és jerk limiteket vegyen figyelembe a rendszer	$\frac{m}{s^2}$ ill $\frac{m}{s^3}$	behavior_planning
Biztonsági távolság	Megadja, hogy milyen távol álljunk meg a kritikus objektumtól	m	plan_long_emergency

3. Architecture

Három node-ot készítettünk: **behavior planner** (viselkedéstervezés), **plan long emergency** (mozgástervezés) és **ctrl long emergency** (mozgásszabályozás).

A node-ok elhelyezkedése a projektstruktúrában:

1. **behavior_planner** – döntéshozatal, kritikus objektum kiválasztása, stratégia meghatározása
2. **plan_long_emergency** – sebességprofil generálása
3. **ctrl_long_emergency** – célsebesség kiszámítása és kiadása a járműnek

3.1. Behavior planner

A behavior_planner node az AEB pipeline döntéshozatali rétege. Feladata, hogy a bejövő scenario és ego adatokból ciklusonként szűrje a releváns objektumokat, kiszámítsa az ütközési paramétereket (TTC, closing speed, távolság), majd ez alapján kiadja a viselkedési döntést és a célterét a downstream tervezők felé.

Interface

Table 9: Subscription

Topic	Típus	Leírás
/scenario	crp_msgs/Scenario	Aktuális scenario, objektumlistával
/ego	crp_msgs/Ego	Ego jármű aktuális kinematikai állapota

Table 10: Public

Topic	Típus	Leírás
/plan/strategy	tier4_planning_msgs/Scenario	Kiadott tervezési scenario
/plan/target_space	crp_msgs/TargetSpace	Célterület a tervezők számára, szűrt objektumokkal
/plan/strategy_behavior	crp_msgs/Behavior	Viselkedési döntés downstream felé

Paraméterek

Table 11: Paraméterek

Paraméter	Típus	Alapértelmezett	Leírás
scenario_topic	string	/scenario	Scenario subscriber topic neve
ego_topic	string	/ego	Ego subscriber topic neve
strategy_topic	string	/plan/strategy	Scenario publisher topic neve
targetSpace_topic	string	plan/target_space	TargetSpace publisher topic neve
behavior_topic	string	"plan/strategy_behavior"	Behavior publisher topic neve
sensitivity	int	1	Fékezési mód (deceleration_mode) értéke a behavior üzenetben
debug_enabled	bool	false	Részletes logolás ki/be kapcsolása

Algoritmus

Timer callback

A node 50 Hz-en fut. Minden ciklusban validálja a bemeneti adatokat, feldolgozza az objektumlistát, szűri a releváns elemeket, majd publikálja a döntést és a célterületet.

A timer a konstruktorban kerül létrehozásra:

```
timer_pub = this->create_wall_timer(
    std::chrono::milliseconds(20),
    std::bind(&BehaviorPlanner::timerCallback, this)
);
```

A callback 20 ms-enként hívódik meg, függetlenül attól, hogy érkezett-e új adat. Az adatokat a last_ego és last_scenario tag változók tárolják, amelyeket a subscriber callbackek töltenek fel.

Scenario érvényesség kezelése

A timerCallback első lépése meghatározni, melyik scenario adatot dolgozza fel az adott ciklusban. Három eset lehetséges:

- 1. **Friss scenario érkezett** – a last_scenario érvényes, ezt használja a node, majd nullázza, hogy a következő ciklusban ne dolgozza fel újra.
- 2. **Nincs friss scenario, de az utolsó érvényes még felhasználható** -a node extrapolálja az utolsó ismert állapotot a calcMissingObject() segítségével. Ez legfeljebb MAX_MISSING_CYCLES (5) cikluson át lehetséges. A dt értéke az utolsó érvényes scenario időbélyege és az aktuális ciklus ideje közötti különbség.
- 3. **A kiesés meghaladja a limitet** – a node üres TargetSpace üzenetet publikál, és visszatér. Downstream rendszer nem kap érvénytelen adatot.

```
if (last_scenario) {
    working_scenario = last_scenario;
    last_scenario = nullptr;
}
else if (last_valid_scenario_ && (missing_scenario_cycles_ <= MAX_MISSING_CYCLES)) {
    missing_scenario_cycles_++;
    double dt = (this->now() - last_valid_scenario_time_).seconds();
}
else {
```

```

    // üres TargetSpace publikálás, return
}
}

```

A `missing_scenario_cycles_` számlálót a `scenarioCallback` nullázza vissza minden érkező érvényes üzenetnél.

Objektumszűrés

A `timerCallback` a `scenario` objektumait két listában dolgozza fel: `local_obstacles` (statikus akadályok) és `local_moving_objects` (mozgó objektumok). Mindkét listán ugyanaz a szűrési logika fut.

Írányvizsgálat – Az ego haladási irányával ellentétes oldalon lévő objektumok kizárásra kerülnek. Ez az ego hosszirányú sebességének (`ego_vx`) előjele alapján dől el:

```

if (obstacle_x < 0.0 && ego_vx > 0.0) continue; // ego előre megy, objektum mögötte
if (obstacle_x > 0.0 && ego_vx < 0.0) continue; // ego hátra megy, objektum előtte

```

Relevancia kritérium – Egy objektum akkor kerül a `relevant_*` listába, ha teljesül az alábbiak valamelyike:

- Távolsága kisebb mint 100 m (`distance <= limit`), vagy
- TTC kisebb mint 2 s és `closing_speed` nagyobb mint 0,5 m/s

```

if ((distance <= limit) || (ttc < 2.0 && closing_speed > 0.5)) {
    relevant_obstacles.push_back(current_obstacle);
}

```

A szűrt listák az `out_targetSpace` üzenetbe kerülnek, és a `pub_targetSpace`-en kerülnek publikálásra.

TTC számítás – `calcTTC()`

Minden objektumra a `node` kiszámítja a Time-to-Collision értéket. Az eredmény egy `Result` struktúrában kerül visszaadásra:

```

struct Result {
    double ttc;
    double closing_speed;
    double dist_actual;
    double ego_v;
};

```

A számítás menete:

1. Az ego abszolút sebessége a `twist.twist.linear.x/y` komponensekből kerül kiszámításra.
2. A relatív sebességvektor az ego és az objektum sebességvektorainak különbsége.
3. A `closing_speed` az objektum irányába mutató egységvektor és a relatív sebességvektor skaláris szorzata – azt méri, hogy a két jármű milyen gyorsan közeledik egymáshoz.
4. A TTC a távolság és a `closing_speed` hányadosa. Ha a `closing_speed` kisebb mint 0,1 m/s, a TTC értéke $1 \cdot 10^{-6}$ s (gyakorlatilag végtelen – nem fenyeget ütközés).

```

double closing_speed = 0.0;
if (dir_len > 0.001) {
    object_x /= dir_len;
    object_y /= dir_len;
    closing_speed = rel_vx * object_x + rel_vy * object_y;
}
double ttc = (closing_speed > 0.1) ? distance / closing_speed : 1e6;

```

A 0.001-es küszöb a nullával való osztás ellen véd, ha az objektum az ego pozíciójával közel azonos helyen lenne.

Behavior üzenet publikálása

A döntési ciklus végén a node közzéteszi a viselkedési döntést. A deceleration_mode értékét a sensitivity paraméterből olvassa be, ezért az futásidőben is módosítható:

```
crp_msgs::msg::Behavior behavior_msg;  
behavior_msg.deceleration_mode.data = static_cast<uint8_t>(  
    this->get_parameter("sensitivity").as_int()  
);  
pub_behavior_->publish(behavior_msg);
```

Belső adatstruktúrák

A node az objektumokat négy külön vektorban tartja nyilván, amelyek a timerCallback feldolgozása során töltődnek fel: A relevant_* vektorok kerülnek az out_targetSpace üzenetbe.

Table 12: FSR

Változó	Leírás
objects	Az összes mozgó objektum a working scenarioból
obstacles	Az összes statikus akadály a working scenarioból
relevant_objects	Szűrt mozgó objektumok (irány + relevancia)
relevant_obstacles	Szűrt statikus akadályok (irány + relevancia).

Biztonsági megfontolások

A node több rétegű védelmet alkalmaz:

- Hiányzó ego adat: Ha last_ego még nem érkezett meg, a timerCallback visszatér publikálás nélkül.
- Scenario kiesés: Legfeljebb MAX_MISSING_CYCLES (5) cikluson át extrapolálja az utolsó érvényes scenariót; ezt meghaladón üres TargetSpace-t publikál és visszatér.
- Irányvizsgálat: Az ego mögött lévő objektumok kizárásra kerülnek, elkerülve az irreleváns beavatkozásokat.
- Nullával való osztás: A closing speed számításakor $dir_len > 0.001$ küszöb védi a számítást; a TTC csak $closing_speed > 0.1$ esetén kap valódi értéket.
- Kinematikai extrapoláció: A calcMissingObject() csak az utolsó érvényes állapot alapján becsül, nem vesz fel külső adatot.

Követelmény lefedettség:

- TSR-02 – releváns objektumok azonosítása: relevant_obstacles / relevant_objects szűrés
- TSR-03 – TTC: számítás: calcTTC() implementáció
- TSR-05 – sensitivity alapú döntés: sensitivity paraméter - deceleration_mode
- SWE-01 – determinisztikus futás fix ciklusidőn: 50 Hz-es create_wall_timer
- SWE-03 – nullával való osztás elleni védelem: $dir_len > 0.001$ és $closing_speed > 0.1$ küszöbök
- SWE-04 – adatkiesés tűrés: calcMissingObject() + MAX_MISSING_CYCLES limit + üres TargetSpace fallback

3.2. Plan long emergency

A plan_long_emergency node az AEB (Autonomous Emergency Braking) rendszer része. Feladata, hogy a behavior_planner által kiválasztott kritikus objektum és az ego jármű állapota alapján egy helyes lassulási sebességprofilot generáljon, amelyet aztán a ctrl_long_emergency hajt végre.

Interface

Table 13: Subscription

Topic	Típus	Leírás
/ego	crp_msgs/Ego	Ego jármű aktuális állapota (pozíció, sebesség, gyorsulás)
plan/target_space	crp_msgs/TargetSpace	Kritikus objektum(ok) listája
plan/strategy_behavior	crp_msgs/Behavior	Aktív lassulási mód (0/1/2)

Table 14: Public

Topic	Típus	Leírás
/plan/longEmergency/trajectory	autoware_planning_msgs/Trajectory	Generált TrajectoryPoint sorozatként

Paraméterek

Table 15: Paraméterek

Paraméter	Típus	Alapértelmezett	Leírás
safety_distance	double	5.0 m	Minimális megállási távolság az objektumtól
prediction_horizon	double	2.5 s	Trajectory előretekintési idő
ego_topic	string	/ego	Ego topic neve
target_topic	string	plan/target_space	Célhalmaz topic neve
behavior_topic	string	plan/strategy_behavior	Behavior topic neve
output_topic	string	plan/longEmergency/trajectory	Kimenet topic neve
debug_enabled	bool	false	Részletes logolás ki/be kapcsolása

A deceleration_mode értéke határozza meg a lassulási korlátokat:

Table 16: Subscription

Mód	max_accel (m/s^2)	max_jerk (m/s^3)
0	1.5	2.0
1	2.5	4.0
2	4.0	6.0

Algoritmus

Timer callback

A node 50 Hz-en fut (20 ms-es ciklusidő). Minden ciklusban először ellenőrzi, hogy van-e érvényes ego és target_space adat - ha nincs, a függvény visszatér és nem publikál semmit. Ha a relevant_objects lista nem üres, beolvassa a behavior üzenetből az aktív módot (ha ismeretlen mód érkezik, visszaesik az 1-es módra), majd meghívja a generateTrajectory függvényt és publikálja a kapott trajektóriát.

Trajectory generálás

generateTrajectory():

- **1. Szükséges lassulás kiszámítása.** Az objektum távolságából és az ego sebességéből a kinematikai egyenletből ($v^2 = v_0^2 + 2 \cdot a \cdot s$, ahol $v = 0$) adódik a szükséges lassulás:

$$distance = \sqrt{(x_{obj} - x_{ego})^2 + (y_{obj} - y_{ego})^2} \quad (1)$$

$$s_{stop} = \max(0.1, distance - distance_{safety}) \quad (2)$$

$$a_{req} = -\frac{v_{ego}^2}{2 \cdot s_{stop}} \quad (3)$$

- **2. Korlát alkalmazása.** A kiszámított lassulást a mód szerinti maximális gyorsulással korlátozzuk:
`if ( a_req < -lims.max_accel ) a_req = -lims.max_accel;`
- **3. Sebességprofil.** A trajectory pontjait $\Delta t = 0.1$ s lépésközzel integráljuk a prediction_horizon időtávig. Ha a sebesség nullára csökken, a ciklus leáll.

```
curr_v += a_req * dt;  
if ( curr_v < 0 ) curr_v = 0;  
curr_s += curr_v * dt;
```

Minden TrajectoryPoint tartalmaz: longitudinális célsebességet, gyorsulást, pozíciót ($x = x_{ego} + s_{curr}$, $y = y_{ego}$ változatlan), valamint a trajektória kezdetétől mért időbélyeget.

Biztonság

Ha nincs érvényes ego vagy target_space adat, a node hallgat - nem küld érvénytelen trajectoryt. Ez szándékos tervezési döntés: a downstream szabályzó az utolsó érvényes trajectoryval dolgozik tovább, amíg nem érkezik friss adat.

- Az s_{stop} értéke minimum 0.1 m-re korlátozott, így nullával való osztás nem fordulhat elő.
- Negatív sebesség tiltva van a profilban.
- Ismeretlen deceleration_mode esetén a közepes (1-es) módra esik vissza a rendszer.

Követelmény lefedettség

- FSR-01 – sebességprofil generálása accel és jerk korláttal: a_{req} limitálása mode_limits_ alapján.
- SWE-01 – determinisztikus futás fix ciklusidőn: 50 Hz-es create_wall_timer
- SWE-02 – lebegőpontos stabilitás: $\max(0.1, s_{stop})$, $curr_v \geq 0$ guard
- SR-01 – szenzorvaliditás-ellenőrzés: $!last_ego_|| !last_target_guard$

3.3. Control long emergency

A ctrl_long_emergency node az AEB rendszer mozgásszabályzó rétege. Feladata, hogy a plan_long_emergency által generált trajectoryból ciklusonként egy konkrét target speedet számítson ki, és azt kiadja a jármű felé. A node a trajectory első pontját veszi alapul, és az aktuális ego sebességből kiindulva lineárisan interpolál a kívánt sebességig a rendelkezésre álló idő alatt.

Interface

Table 17: Subscription

Topic	Típus	Leírás
/plan/longEmergency/trajectory	autoware_planning_msgs/Trajectory	Sebességprofil a mozgástervezőtől
/ego	crp_msgs/Ego	Az ego jármű aktuális sebessége

Table 18: Public

Topic	Típus	Leírás
/control/command/control_cmd	autoware_control_msgs/Control	Kiszámított target speed jármű felé

Paraméterek

Table 19: Paraméterek

Paraméter	Típus	Alapértelmezett	Leírás
debug_enabled	bool	false	Részletes logolás ki/be kapcsolása

Algoritmus

Timer callback

A node 50 Hz-en fut. Minden ciklusban először validálja a bemeneti adatokat, majd kiszámítja a következő ciklusra vonatkozó target speedet és publikálja azt.

A timer a konstruktorban kerül létrehozásra, 20 ms-es ciklusidővel:

```
m_timer_ = this->create_wall_timer(  
    std::chrono::milliseconds(20),  
    std::bind(&CtrlLongEmergency::timerCallback, this)  
);
```

A callback minden 20 ms-ben meghívódik, függetlenül attól, hogy érkezett-e új adat a subscriberekből. Az adatok tárolása olyan változóknak történik (m_egoVelocity, m_trajectoryVelocity, m_trajectoryTime), amelyeket a callbackek töltenek fel.

Validáció

A timerCallback három feltételt ellenőriz:

- Az ego sebesség nem nulla (m_egoVelocity)
- A trajectory sebesség értelmezett – nem NaN és nem hiányzó
- A trajectory időbélyege érvényes: $0 < m_trajectoryTime \leq 4.0$ s

Ha bármelyik feltétel nem teljesül, a ciklus visszatér és nem publikál semmit. A kódban ez a következőképpen jelenik meg:

```
if (!m_egoVelocity) {return;}
if (!m_trajectoryVelocity && m_trajectoryVelocity != 0) { return; }
if (!(m_trajectoryTime > 0.0) || m_trajectoryTime > 4.0) { return; }
```

A trajectory sebességének ellenőrzése azért kettős feltétel, mert a float típusú m_trajectoryVelocity esetén a nullával való egyenlőség vizsgálat külön szükséges: a !m_trajectoryVelocity kifejezés 0.0 esetén is true-t ad vissza, ezért a && m_trajectoryVelocity != 0 kizárja, hogy a nulla értékű (megállt objektum) trajektóriát érvénytelennek tekintse a rendszer.

Target speed számítása

A target speed számítása lineáris interpolációval történik az aktuális ego sebesség és a trajectory által előírt target speed között, a rendelkezésre álló idővel számolva:

```
cycles          = m_trajectoryTime / 0.02
speedPerCycle   = (m_egoVelocity - m_trajectoryVelocity) / cycles
targetSpeed     = m_egoVelocity - speedPerCycle
```

Ahol:

- m_trajectoryTime – a trajectory első pontjának time_from_start értéke másodpercben
- m_egoVelocity – az ego jármű aktuális sebessége m/s-ban
- m_trajectoryVelocity – a trajectory első pontjának target speede m/s
- cycles – hány 20 ms-es ciklus alatt kell elérni a target speedet
- speedPerCycle – ciklusonként mekkora sebességváltozás kell

A számítás lényege: ahelyett hogy azonnal a trajectory target speedére ugrana, a node egyenletes lépésekben közelíti azt meg a rendelkezésre álló idő alatt, ettől gördülékenyebb lesz a beavatkozás folyamata.

Implementáció a timerCallback-ben:

```
double cycleTimeSec = 0.02;
double cycles = m_trajectoryTime / cycleTimeSec;
double speedPerCycle = (m_egoVelocity - m_trajectoryVelocity) / cycles;
double targetSpeed = m_egoVelocity - speedPerCycle;
if (targetSpeed < 0.0) targetSpeed = 0.0;
controlMsg.longitudinal.velocity = targetSpeed;
if (!std::isnan(controlMsg.longitudinal.velocity)) m_pubControl->publish(controlMsg);
```

A m_trajectoryTime értékét a trajectoryCallback tölti fel a trajektória első pontjának time_from_start mezőjéből, amely azt jelzi, mennyi idő alatt kell az első célsebességet elérni. Minél rövidebb ez az idő, annál nagyobb lesz a speedPerCycle – azaz gyorsabb lassítást hajt végre a rendszer.

Ha a kiszámított targetSpeed negatív lenne, nullára korlátozzuk – a jármű nem mehet hátrafele. Ha NaN-t kapnánk a publikálás elmarad.

A trajectoryCallback a bejövő trajektória első pontjából olvassa ki az adatokat, és eltárolja őket tag változóiban. Az időbélyeg sec és nanosec részekből áll össze egyetlen double értékke:

```

m_trajectoryVelocity = msg->points[0].longitudinal_velocity_mps;
m_trajectoryTimeSec   = msg->points[0].time_from_start.sec;
m_trajectoryTimeNanosec = msg->points[0].time_from_start.nanosec;
m_trajectoryTime = m_trajectoryTimeSec + m_trajectoryTimeNanosec * 1e-9;

```

Az egoCallback egyszerűbb: csupán a twist.twist.linear.x mezőből veszi ki az aktuális hosszirányú sebességet m/s-ban, és eltárolja m_egoVelocity-ban.

Biztonság

A node több rétegű védelmet alkalmaz. Az érvénytelen vagy hiányzó bemeneti adatok esetén nem publikál semmit, így a downstream rendszer nem kap helytelen parancsot. A trajectory időkorlátja (max 4 s) megakadályozza, hogy elavult vagy irreálisan hosszú tervből számítson sebességet. A negatív sebesség kizárása és a NaN ellenőrzés numerikus stabilitást biztosít.

Követelmény lefedettség

- TSR-07 – target speed kimenet: controlMsg.longitudinal.velocity publikálása
- SWE-01 – determinisztikus futás fix ciklusidő: 50 Hz-es create_wall_timer
- SWE-02 – numerikus stabilitás: NaN guard, negatív sebesség kizárása
- TSR-01 – validitásellenőrzés: ego és trajectory adat ellenőrzése minden ciklusban

4. Function Description

behavior_planner node

A behavior_planner node a rendszer döntéshozatali rétege. Feliratkozik az ego jármű állapotára (/ego) és a szenzor által kiadott /scenario-ra, majd minden ciklusban meghatározza, hogy a látótérben lévő objektumok közül melyek relevánsak a vészfékező rendszer szempontjából. Az eredményt a plan/target_space topicra publikálja, ahol a mozgástervező node veszi át. Emellett a plan/strategy_behavior topicra is küld egy Behavior üzenetet, amely az érzékenységi szintet állítja és meghatározza, hogy a mozgástervező milyen lassítást alkalmazzon.

A nodeban a scenario_topic, ego_topic, strategy_topic, targetSpace_topic és behavior_topic paraméterek lehetővé teszik a topic nevek futás közbeni megváltoztatását. A sensitivity paraméter (0, 1 vagy 2 értékű egész) az érzékenységi szintet állítja be, amelyet a behavior_planner közvetlenül átad a mozgástervezőnek a Behavior üzenet deceleration_mode mezőjén keresztül. A debugEnabled paraméter ki-be kapcsolja a részletes log üzeneteket, ami debugolás során hasznos, éles üzemből pedig kikapcsolható a teljesítmény növeléshez.

Adatkiesés kezelése(H-03)

Az egyik legfontosabb biztonsági funkció a szenzoradat kiesés kezelése (HARA H-03). A rendszer az utolsó érvényes scenario üzenetet és ennek időpontját eltárolja (last_valid_scenario_, last_valid_scenario_time_). Ha a következő ciklusban nem érkezik friss scenario adat, a node nem leáll, hanem kinematikai extrapolációval becsüli meg az objektumok aktuális helyzetét.

Az extrapolációt a calcMissingObject függvény végzi, amely minden objektumra külön-külön alkalmazza a kinematikai mozgásegyenleteket. A pozíciót az alábbi összefüggés szerint számítja:

$$x(t) = x_0 + v_0 \cdot dt + \frac{1}{2} \cdot a_0 \cdot dt^2 \quad (4)$$

$$y(t) = y_0 + v_{y0} \cdot dt + \frac{1}{2} \cdot a_{y0} \cdot dt^2 \quad (5)$$

A sebességet pedig az egyenletes gyorsulás tétele alapján frissíti:

$$vx(t) = vx_0 + ax_0 \cdot dt \quad vy(t) = vy_0 + ay_0 \cdot dt \quad (6)$$

ahol dt a kiesett idő másodpercben mérve, vx_0 és vy_0 az utolsó ismert sebességkomponensek, ax_0 és ay_0 az utolsó ismert gyorsuláskomponensek. Az extrapoláció legfeljebb MAX_MISSING_CYCLES, azaz 5 cikluson keresztül – összesen 100 ms-ig – folytatódhat. Ez megfelel az FSR-04 és TSR-02 követelményeknek, amelyek 5 ciklusnyi adatki-maradás esetén is robusztus működést írnak elő. Az 5 ciklus elteltével a node üres TargetSpace üzenetet publikál és visszatér, ezzel jelezve a downstream rendszernek, hogy nem áll rendelkezésre megbízható információ.

Objektumszűrés és time to collision számítás

A timerCallback fő feladata az, hogy a jelenetet feldolgozza és meghatározza, mely objektumok relevánsak a beavatkozás szempontjából. A scenario üzenet két objektumlistát tartalmaz: a local_obstacles (statikus akadályok) és a local_moving_objects (mozgó objektumok) tömböket. Mindkét listán azonos szűrés logika fut végig.

Az első szűrés feltétel az irányfüggő kizárás: ha az ego pozitív irányban halad ($ego_vx > 0$), akkor csak az ego előtt lévő objektumok ($object_x > 0$) kerülnek figyelembe, ha negatív irányban megy, akkor csak a mögötte lévők. Ez megakadályozza, hogy a rendszer a háta mögötti objektumokra feleslegesen reagáljon, és lefedi az FSR-02 tolatós objektum kezelési követelményét is.

A távolság alapú szűrés ezután a 100 méteres korlátot vizsgálja, amit az euklideszi távolság alapján számít:

$$distance = \sqrt{object_x^2 + object_y^2} \quad (7)$$

Ezen kívül a time to collision (Time to collision, ütközésig hátralévő idő) alapú szűrés is fut: ha az objektumhoz számolt time to collision kisebb 2 másodpercnél és a zárási sebesség nagyobb 0.5 m/s-nál, az objektum szintén relevánsnak minősül, és bekerül a relevant_objects vagy relevant_obstacles listába. A calctime to collision függvény a relatív sebességből és távolságból számítja a time to collision értéket, figyelembe véve a closing_speed-et, vagyis azt, hogy az ego és az objektum milyen ütemben közelít egymáshoz.

A szűrt listák ezután bekerülnek az out_targetSpace üzenetbe, amelyet a node a plan/target_space topicra publikál. Ezzel a behavior_planner átadja a mozgástervezőnek pontosan azokat az objektumokat, amelyekre reagálni kell, leszűrve a nem releváns zajt.

plan_long_emergency node

A plan_long_emergency node a rendszer mozgástervező rétege. Feliratkozik a plan/target_space, az /ego és a plan/strategy_behavior topicokra. Feladata, hogy a kritikus objektumok adatai és az ego jármű állapota alapján egy trajec-toryt generáljon, ezzel meghatározva, hogy mikor milyen sebességgel kell haladnia. A generált trajectory egyetlen TrajectoryPoint-ból áll: ez tartalmazza a célsebességet és az elérési időt.

A node három különböző lassulási szintet támogat, amelyek a behavior_planner által küldött deceleration_mode értékétől függenek. Az egyes szintekhez tartozó gyorsulási és jerk korlátok (max_accel, max_jerk) paraméterként konfigurálhatók, így a rendszer különböző érzékenységi szinteken különböző erejű beavatkozást végezhet. Ez megfelel az FSR-05 soft limit módok követelményének.

Time to collision számítás

A node legfontosabb algoritmus a calculate_universal_time to collision függvény, amely mind az egyenletes sebességű, mind a gyorsuló-lassuló mozgást kezeli.

A függvény először kiszámítja a relatív sebességet és a relatív gyorsulást:

$$rel_v = v_ego - v_target \quad rel_a = a_ego - a_target \quad (8)$$

Ha a relatív gyorsulás elhanyagolható (kisebb mint egy konfigurálható küszöbérték, alapértelmezetten 10^{-5}), a függvény a konstans sebességű közelítésre esik vissza:

$$time_to_collision = distance / rel_v \quad (9)$$

(ha $rel_v > 0$, különben nincs ütközés)

Ha a relatív gyorsulás szignifikáns, a függvény a kinematikai egyenlet másodfokú megoldásával határozza meg az ütközés időpontját. Az egyenlet a következő fizikai gondolaton alapul: az objektum és az ego jármű egymáshoz képesti mozgása egy egyenletesen gyorsuló folyamatnak tekinthető, és az ütközés abban a pillanatban következik be, amikor a köztük lévő relatív távolság nullává válik:

$$dt = distance - rel_v \cdot t - \frac{1}{2} \cdot rel_a \cdot t^2 = 0 \quad (10)$$

Ebből a másodfokú egyenletből a diszkrimináns:

$$\Delta = rel_v^2 + 2 \cdot rel_a \cdot distance \quad (11)$$

Ha a diszkrimináns negatív, nincs valós megoldás, ami azt jelenti, hogy az objektum és az ego jármű kinematikailag elkerülik egymást – ebben az esetben a függvény std::nullopt-ot ad vissza. Ha a diszkrimináns nem negatív, a két lehetséges időpont:

$$t_1 = (-rel_v + \sqrt{\Delta}) / rel_a \quad (12)$$

$$t_2 = (-rel_v - \sqrt{\Delta}) / rel_a \quad (13)$$

A függvény a két pozitív megoldás közül a kisebbet választja, hiszen ez az első jövőbeli ütközési pillanat. Ez a megközelítés FSR-02 teljesítését biztosítja, lefedve az álló, lassuló, konstans sebességű és tolató objektum eseteket egyaránt.

Tracjectory generálás

A generateTrajectory függvény az összes releváns objektum közül a legkisebb time to collision-vel rendelkezőt keresi meg. Ez az objektum jelenti a legfontosabb veszélyt, így a trajectory erre az objektumra optimalizálódik. A keresés az összes relevant_objects elemen végigiterál, és minden objektumra meghívja a calculate_universal_time_to_collision függvényt. A legkisebb pozitív time to collision értéket adó objektum távolsága és time to collision értéke kerül felhasználásra a trajectory felépítésekor.

A generált trajectory egyetlen pontból áll, amelynek célsebessége nulla és az időbélyege a legkisebb time to collision értéknek felel meg. Ez azt közli a szabályzóval, hogy a rendszer megállás, és erre mennyi idő áll rendelkezésre. A szabályzó ebből az információból tudja lineárisan interpolálni a szükséges célsebességet minden 20 ms-es ciklusban. Ha nem érkezik target adat, vagy üres a relevant_objects lista, a node nem publikál trajectoryt – ezt a védvonal az onTimer elején lévő visszatérési feltétel biztosítja:

```
if (!last_ego_ || !last_target_) return;
```

Ez megfelel a TSR-01 követelménynek, amely előírja, hogy minden ciklusban ellenőrizni kell az adatok érvényességét.

ctrl_long_emergency node

A ctrl_long_emergency node a rendszer szabályzó rétege, és egyben a pipeline utolsó tagja. Feladata, hogy a mozgástervező által előállított trajectoryból ciklusonként egy végrehajtható célsebességet számítson, és azt az /control/command/control_cmd topicra publikálja. Ez az érték kerül közvetlenül a fékrendszerhez.

A node az /ego topicra is feliratkozik, hogy az aktuális ego sebességet minden ciklusban újraolvassa. A trajectoryCallback a trajectory első pontjából veszi ki a célsebességet és az időbélyeget, az egoCallback pedig az ego sebesség értékét tárolja el. Mindkét callback pusztán adattároló szerepet tölt be, a tényleges számítás a 20 ms-enkénti timerCallback-ben zajlik.

Célsebesség számítása – lineáris interpoláció

A célsebesség számításának alapötlete az, hogy a rendszer ne hirtelen, azonnal a nulla sebességre váltson, hanem fokozatosan, lineárisan lassítson a rendelkezésre álló idő alatt. A számítás a következő lépésekből áll: Először meghatározzuk, hogy hány 20 ms-es ciklus áll rendelkezésre a célsebesség eléréséhez:

```
cycles = m_trajectoryTime / 0.02
```

Majd kiszámítjuk, hogy ciklusonként mekkora sebességváltozás szükséges az ego és a trajectory célsebessége között:

```
speedPerCycle = (m_egoVelocity - m_trajectoryVelocity) / cycles
```

Az aktuális ciklusra vonatkozó célsebesség:

```
targetSpeed = m_egoVelocity - speedPerCycle
```

Mivel a m_trajectoryVelocity értéke nulla, a speedPerCycle értéke egyenlő az ego sebesség elosztva a rendelkezésre álló ciklusok számával. Minél rövidebb a m_trajectoryTime, annál nagyobb speedPerCycle adódik, és annál gyorsabban lassul a rendszer. Ez automatikusan kezeli a vészfékezés intenzitását a szituáció súlyosságától függően.

Validáció és biztonsági korlátok

A timerCallback három egymást követő feltételt ellenőriz, mielőtt bármilyen számítást végezne. Az első az ego sebesség meglétét vizsgálja: ha `m_egoVelocity` nulla értékű és korábban még nem érkezett adat, a ciklus visszatér. A második feltétel a trajectory sebesség értelmességét ellenőrzi, az alábbi kettős feltétellel:

```
if (!m_trajectoryVelocity && m_trajectoryVelocity != 0) return;
```

Ennek az a célja, hogy elkülönítse a hiányzó adatot (trajectory hiánya) a szándékosan nulla célsebességű esettől. Az egyszerű `!m_trajectoryVelocity` kifejezés ugyanis 0.0 esetén is igaz lenne, ami félrevezető lehetne. A harmadik feltétel a trajectory időbélyegét vizsgálja: az időnek pozitívnak kell lennie, és nem haladhatja meg a 4 másodpercet. Ez a korlát megakadályozza, hogy a rendszer elavult vagy irreálisan messze lévő trajectory alapján számoljon. A 4 másodperces felső korlát az FSR-03 kritériumot teljesíti: ennél hosszabb time to collision esetén nincs szükség beavatkozásra.

A számítás végén két további biztonsági korlát is érvényesül. Az egyik a negatív sebesség kizárása, amely megakadályozza, hogy a jármű véletlenül hátramenetbe kapcsoljon:

```
if (targetSpeed < 0.0) targetSpeed = 0.0;
```

A másik a NaN ellenőrzés, amely numerikus hibák esetén megakadályozza az érvénytelen parancs kiadását:

```
if (std::isnan(controlMsg.longitudinal.velocity)) return;
```

Ez az SWE-02 követelmény teljesítését biztosítja, amely a lebegőpontos numerikus stabilitást írja elő.

Rendszerszintű működés és data stream

A három node egymást követve kerül feldolgozásra fel 20 ms-ként. Az `/ego` és `/scenario` topic publikálódnak, a `behavior_planner` feldolgozza a jelenetet, kiszűri a releváns objektumokat, és kiadja a `plan/target_space` és `plan/strategy_behavior` üzeneteket. A `plan_long_emergency` a `target_space`-ből veszi az objektumokat, time to collision számítással meghatározza a legközelebbi veszélyt, és egyetlen trajectory pontot ad ki, amely tartalmazza a célsebességet (0) és a rendelkezésre álló időt. A `ctrl_long_emergency` ebből lineáris interpolációval számítja a következő ciklusra érvényes célsebességet, és azt elküldi a jármű felé.

Az összes paramétert – `sensitivity`, `safety_distance`, `prediction_horizon`, `mode_limits`, `debug_enabled` – a ROS2 paraméterrendszer kezeli, így azok futás közben is megváltoztathatók, launch fájlból betölthetők. Ez megfelel a TSR-04 kalibrációs fájl követelménynek.

Követelmény lefedettség összefoglalása

Table 20: Követelmények-összefoglalása

Követelmény ID	Leírás	Lefedő komponens
FSR-01	Lassulási profil gyorsulás és jerk korláttal	plan_long_emergency – mode_limits_
FSR-02	Álló, lassuló, konstans, tolató objektum kezelése	plan_long_emergency - calculate_universal_time to collision
FSR-03	Elkerülhető / nem elkerülhető ütközés jelzése	behavior_planner – time to collision alapú szűrés
FSR-04	Robusztusság 5 ciklusnyi adatkimaradásra	behavior_planner – calcMissingObject extrapoláció
FSR-05	3 Soft Limit mód konfigurálható korlátokkal	plan_long_emergency – mode_limits_map
TSR-01	Szenzoradat validitás-ellenőrzés	Mindhárom node – bemeneti guard feltételek
TSR-02	Adatkimaradás extrapolációval max. 5 ciklusig	behavior_planner – missing_scenario_cycles_
TSR-05/06	Interfész: pozíció, sebesség, gyorsulás	crp_msgs üzenettípusok
TSR-07	Kimenet: célsebesség	ctrl_long_emergency – longitudinal.velocity
SWE-01	Determinisztikus futás fix ciklusidőn	plan_long_emergency – mode_limits_
SWE-02	Numerikus stabilitás	ctrl_long_emergency – NaN guard, negatív korlát
SWE-03	Üres objektumlista kezelése	plan_long_emergency – relevant_objects ellenőrzés
SWE-06	Trace log diagnosztikai célra	ctrl_long_emergency – NaN guard, negatív korlát

5. Validation

5.1. Simulator

Give details of the simulator.

5.2. Test cases

Your test definitions in table format.

5.3. Results

Graphical and numerical results of the tests.

References